Product Kernel Perspective Spaces: Predicting Benchmark Scores from Sparse, Unpaired Model Evaluations

Anonymous

Abstract

Evaluating language models is expensive, so benchmark coverage is sparse and noisy: each model is run on only some tasks, only some queries within a task, and each score is a noisy average over those queries. Methods that complete the (model $\times$ task) score matrix—matrix completion, item-response theory—use only each cell's *aggregate* score. We use *item-level* information instead: a cell's individual responses, not just its score. We introduce the **Product Kernel Perspective Space (PKPS)**, which embeds each model from its responses through a product kernel $k_Q \cdot k_R$ that compares responses to *similar* queries, not only identical ones; it recovers the Data Kernel Perspective Space as the bandwidth $\sigma \to 0$ and stays defined when models answer different queries. Ensembling this embedding with the available score signal—a cell's own sample mean where measured, matrix completion where not—makes efficient benchmarking and benchmark completion two ends of one problem. On a heterogeneous 18-task, 93-model suite, the ensemble beats the best score-only predictor across task coverage, queries per cell, and cohort size; the embedding survives unpaired queries where DKPS and IRT collapse, and rescues small cohorts where low-rank completion fails for lack of rows.

1 Introduction

Comprehensive evaluation of language models is a growing bottleneck: leaderboards track hundreds of models across dozens of benchmarks, yet no model is run on everything. The resulting (model $\times$ task) score matrix is *doubly sparse*. *Across tasks*, a model is evaluated on a subset of benchmarks (missing tasks). *Within a task*, evaluation may use only a subset of queries to save cost (missing queries). Predicting the missing entries—a model's score on a task it has not been (fully) run on—would let practitioners reuse cached evaluations instead of paying to re-run them [Polo et al., 2024, Vivek et al., 2024, Zeng and Papailiopoulos, 2026].

The Data Kernel Perspective Space (DKPS) [Helm et al., 2026] is a black-box approach: it embeds each model from its query responses, places models in a low-dimensional Euclidean space via classical multidimensional scaling, and regresses benchmark scores in that space. DKPS is query-efficient and competitive with item-response theory. However, it has a structural limitation: the distance between two models is the average squared difference of their responses *to the same queries*, so DKPS requires a *paired* design. When models answer different queries—the common case in real, organically-assembled leaderboards—DKPS has nothing to compare and the information in those unpaired responses is thrown away.

We introduce the **Product Kernel Perspective Space (PKPS)**. The key change is to compare responses to *similar* queries, not only identical ones, by weighting each response-pair comparison with a query kernel. This (i) lets every unpaired response contribute, and (ii) contains DKPS as the special case where the query kernel is the identity. The broader thesis is that PKPS supplies an *item-level embedding* signal that score-completion baselines ignore, and that combining it with the available *item-level score* signal predicts benchmark performance more efficiently than either alone. Our contributions:

- **Methodological and theoretical.** We extend DKPS to the unpaired regime with a product-kernel affinity $k_Q \cdot k_R$ that compares responses to similar queries (Section 3). We show the construction is a strict generalization: it contains paired DKPS as the special case $k_Q = \delta$ and recovers it as the bandwidth $\sigma \to 0$, stays defined when models answer disjoint queries (where paired DKPS is undefined), and interpolates between the two with a single bandwidth, which we select per run by a leak-free cross-validation. We state the expected query-efficiency gain as a conjecture, and treat the embedding as one term of an embedding-$\oplus$-score ensemble.

- **Empirical.** We demonstrate the method's utility and robustness to missing responses on the two ends of one estimation problem—query-efficient evaluation ($m>0$ queries per cell) and matrix completion ($m=0$)—over a heterogeneous 18-task, 93-model suite spanning math, translation, medical QA, and legal classification (Sections 4.1–4.2). The embedding survives unpaired queries where DKPS and IRT collapse and recovers the small-cohort regime where low-rank completion fails for lack of rows; the ensemble beats the best score-only predictor across task coverage, queries per cell, and cohort size, by a margin that grows as evaluation gets sparser, noisier, or smaller-cohort.

2 Related work

Low-dimensional representations of models. Our method contributes to a literature on low-dimensional representations of models. Some methods compare models through their internals: activations on shared inputs [Duderstadt et al., 2023, Huh et al., 2024, Horwitz et al., 2025] or weights [Chen et al., 2025]. Evaluation, however, is black-box: it sees responses, not internals. The Data Kernel Perspective Space (DKPS) embeds models from their responses into a low-dimensional Euclidean space via multidimensional scaling [Torgerson, 1952, Helm et al., 2024]. The representations are consistent and concentrate [Acharyya et al., 2024, 2025], support statistical inference [Helm et al., 2025], and are query-efficient [Helm et al., 2026]. DKPS averages over responses to *shared* queries, so it requires a paired design. PKPS keeps the response-level representation but replaces identity matching with a product kernel, so models that answer different queries remain comparable.

Benchmark prediction. A second line predicts unobserved (model, task) scores so that not every evaluation must be run; efficient benchmarking and matrix completion are its two ends. Efficient benchmarking scores a small item subset and extrapolates. Item-response theory selects items by difficulty and discrimination [Rasch, 1960, Polo et al., 2024, Kipnis et al., 2024]; anchor points cluster items by cross-model agreement [Vivek et al., 2024]; cognitive-scale embeddings [Bean et al., 2025] and reinforcement learning [Li et al., 2024] select items adaptively; and design analyses optimize the cost–reliability tradeoff [Perlitz et al., 2024]. Closer to us, DKPS predicts a held-out score from cached responses [Helm et al., 2026], datamodels predict outputs from inputs [Ilyas et al., 2022], and lifelong benchmarks reuse past evaluations [Prabhu et al., 2024]. Matrix completion instead predicts cells that were never run, exploiting low rank across the score matrix [Candès and Recht, 2009, Mazumder et al., 2010, Koren et al., 2009]; BenchPress finds it is approximately rank-two and completes it in logit space [Zeng and Papailiopoulos, 2026]. These methods use only each cell's *aggregate* score, not its responses. PKPS unifies the two ends—a cell with $m \geq 0$ queries spans efficient benchmarking ($m>0$) and completion ($m=0$)—and adds an item-level embedding signal that we ensemble with matrix completion.

3 The Product Kernel Perspective Space

Models and observations. A *model* is a random mapping $f : \mathscr{Q} \to \mathscr{X}$ from a query space $\mathscr{Q}$ to a response space $\mathscr{X}$; querying f at q returns a response drawn from the distribution $f(q)$. We observe n models $f_1, \dots, f_n$, each run on its own *query set* $Q_i \subseteq \mathscr{Q}$, giving a tuple $\mathbf{Q} = (Q_1, \dots, Q_n)$ that may differ across models or be disjoint; the paired design $Q_1 = \cdots = Q_n$ is the special case. Two fixed, distinct embedding functions map raw objects to vectors: a *response* embedding $\text{embed}_R : \mathscr{X} \to \mathbb{R}^p$ and a *query* embedding $\text{embed}_Q : \mathscr{Q} \to \mathbb{R}^{p'}$ (e.g. different text-embedding models). For $q_j \in Q_i$ we average the embeddings of the r sampled responses, $x_{ij} = \frac{1}{r}\sum_{k=1}^{r} \text{embed}_R\big(f_i(q_j)^{(k)}\big) \in \mathbb{R}^p$, and write $u_j = \text{embed}_Q(q_j)$ for the embedded query. The goal is to predict each model's benchmark score $y_i \in \mathscr{Y}$.

Perspective space. Each method summarizes the n models in an $n \times n$ affinity matrix A (defined below). The squared distance between two models is $D^2(a,b) = A_{aa} + A_{bb} - 2A_{ab}$ (clamped at 0), and their d-dimensional *representations* are the classical-multidimensional-scaling minimizer

$$(\hat{\psi}_1, \ldots, \hat{\psi}_n) = \underset{z_1,\ldots,z_n \in \mathbb{R}^d}{\arg\min} \sum_{i,k=1}^{n} \big(\|z_i - z_k\| - D(i,k)\big)^2, \tag{1}$$

so model i is the point $\hat{\psi}_i \in \mathbb{R}^d$, its *perspective*. Paired DKPS and PKPS differ only in the affinity A.

Paired DKPS. With all n models answering the same m queries ($Q_1 = \cdots = Q_n$), DKPS uses the identity-matched affinity $A_{ab} = \frac{1}{m}\sum_j k_R(x_{aj}, x_{bj})$: response j of model a is paired only with response j of model b, so the design must be paired.

Product kernel. PKPS replaces identity matching with a query kernel k_Q:

$$A_{ab} = \frac{\sum_{j \in Q_a} \sum_{l \in Q_b} k_Q(q_j, q_l)\, k_R(x_{aj}, x_{bl})}{\sum_{j \in Q_a} \sum_{l \in Q_b} k_Q(q_j, q_l)}, \tag{2}$$

where Q_a is the query set of model a. Each term is normalized by its own query-kernel mass, so models that answered different numbers of (or entirely different) queries are still comparable; responses to *similar* queries—even across tasks—contribute and unpaired evaluations are used (Figure 1). Paired DKPS is the special case $k_Q = \delta$, where only $j = l$ survives.

Kernels. The response kernel k_R compares embedded responses, the query kernel k_Q compares embedded queries. Paired DKPS uses a linear response kernel and the identity query kernel $k_Q = \delta$. Throughout our analysis we use a linear or RBF k_R and an RBF query kernel over the embedded queries, $k_Q(q_j, q_l) = \exp(-\|u_j - u_l\|^2/2\sigma^2)$, with bandwidth σ chosen per run (below).

3.1 Theoretical properties of the PKPS

PKPS is a strict generalization of DKPS. **(i) DKPS limit.** Take $k_Q = \delta$, the indicator $\mathbf{1}[q_j = q_l]$. Since the double sum already ranges over $j \in Q_a$, $l \in Q_b$, a term survives only if the two queries are equal *and* that query lies in both sets, $\mathbf{1}[q_j = q_l]\,\mathbf{1}[q_j \in Q_a]\,\mathbf{1}[q_l \in Q_b] = \mathbf{1}[q_j = q_l \in Q_a \cap Q_b]$, so $A_{ab} = \frac{1}{|Q_a \cap Q_b|}\sum_{q \in Q_a \cap Q_b} k_R(x_{aq}, x_{bq})$ —exactly paired DKPS over the shared queries (the same-m-queries case recovers $\frac{1}{m}\sum_j k_R(x_{aj}, x_{bj})$). Moreover, as $\sigma \to 0$ the RBF kernel converges to δ (distinct queries have distinct embeddings), so DKPS is recovered in the small-bandwidth limit. **(ii) Unpaired comparability.** Because each affinity is normalized by its own query-kernel mass $\sum_{jl} k_Q(q_j, q_l)$, two models that answered different—or entirely disjoint—query sets remain comparable, whereas paired DKPS, which sums only over shared queries, is undefined when no query is shared. **(iii) Bandwidth interpolation.** Increasing σ smoothly trades exact query matching for cross-query bridging, so a single scalar controls how much unpaired evidence the perspective uses.

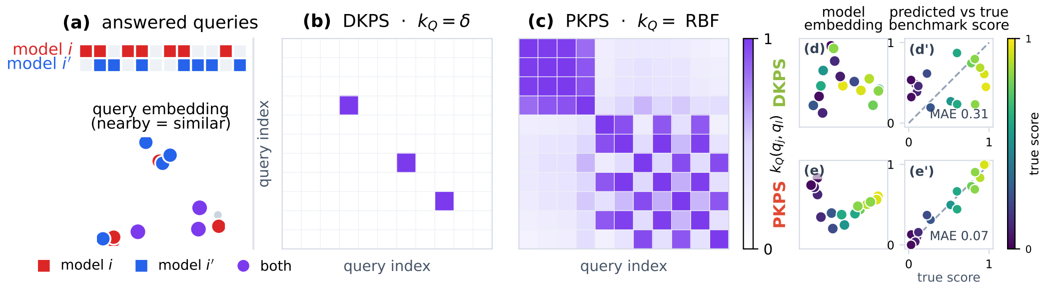

Figure 1: Why the query kernel matters under *unpaired* evaluation: PKPS can use all available information. **Left:** two models, i (red) and i' (blue), each answer their own subset of a shared query pool, shown both as answered-query blocks (top) and as points in query-embedding space (bottom; nearby points are similar queries). The two models share only a few queries. Both kernel panels index the queries by a single universal ordering (shared by rows and columns). **Center (DKPS, $k_Q = \delta$).** The identity query kernel keeps a cell only when the two queries are the *same* ($q_j = q_l$, the diagonal) *and* both models answered it ($q_j \in Q_i \cap Q_{i'}$), so only the few shared queries survive—almost every response is discarded. **Right (PKPS, $k_Q =$ RBF).** The RBF query kernel over the same ordering is a dense, *symmetric* similarity matrix, so responses to *similar* queries—not only identical ones—bridge the two models and every response can contribute. PKPS thus turns the sparse shared-query diagonal DKPS depends on into a dense similarity kernel. **Right block (d, d′, e, e′):** a benchmark-prediction example on a small model population. Each model answers only a few queries from a large pool, so two models rarely answer the *same* query though they answer *similar* ones. Each method embeds the models by classical MDS of the pairwise distance matrix (**d, e**: the 2-D model representations, coloured by true score and rotated so the score gradient runs left-to-right), then predicts each model's full-benchmark score by leave-one-out regression onto that embedding (**d′, e′**: predicted vs. true). Restricted to the rare shared queries, **DKPS** yields a noisy geometry with no clean score axis and predictions scatter off the diagonal (MAE 0.31); **PKPS** bridges all similar queries, so score varies smoothly across its embedding and predictions land on the diagonal (MAE 0.07).

3.2 Inference in the PKPS

Inference is a two-step statistical problem, exactly as in the DKPS [Helm et al., 2026]: embed each model into the perspective space by the scaling of Eq. 1, then regress its benchmark score on the resulting coordinate. Concretely, on a set of reference models with known scores $\{(f_i, y_i)\}$ we fit a decision function $\hat{h} : \mathbb{R}^d \to \mathscr{Y}$ on the pairs $\{(\hat{\psi}_i, y_i)\}$ and predict any model's score as $\hat{y}(f) = \hat{h}(\hat{\psi}(f))$. The only free choices are the query-kernel bandwidth σ and the regressor $\hat{h}$, which we describe next; we then add the score channel that turns the embedding into an ensemble.

Choosing the bandwidth. The optimal σ depends on how paired the data is: small when query overlap is high (recover exact matching), large when overlap is low (bridge across queries). We set it per run by cross-validation over a median-scaled grid $\sigma \in \{0.03, \ldots, 3\} \cdot \text{med}$, where med is the median pairwise query distance, selecting the σ whose perspective best predicts the *observed* scores under leave-one-model-out k-NN. The criterion never touches the held-out scores we report, so the choice is not circular with the evaluation; and because the grid spans down to near-delta, cross-validation *recovers DKPS* when query overlap is high and bridges only when it must. One response-kernel pass produces the entire σ-grid, so tuning adds negligible cost. We use a single regressor (distance-weighted k-NN) and a logit-space, bias-decomposed head throughout.

Estimation pipeline. For each replicate we (i) reduce response and query embeddings by PCA on the *observed* rows only (dimension by the second Zhu–Ghodsi elbow); (ii) form D from Eq. 2; (iii) embed models by classical MDS; (iv) map the embedding to scores with the *same* head as the matrix-completion baseline up to the factor model—logit transform, per-task standardization to unit variance, and a two-way (model + task) additive bias—then a k-NN regression of the residual on the embedding (replacing completion's low-rank factor), predicting every cell (held-out cells directly, observed cells leave-one-out so the estimate never sees its own target). Matching the per-task standardization is important: without it the model offset is dominated by the highest-logit-variance tasks, which systematically biases the embedding on heterogeneous, multi-scale suites.

The embedding-$\oplus$-score ensemble. For each cell we combine the embedding with whichever score signal carries independent information. On a *missing* cell, that is matrix completion of the (noisy) score matrix [Zeng and Papailiopoulos, 2026]; we blend it with PKPS using a weight fit by held-out cross-validation against the observed scores. On an *observed* cell, the cell's own sample mean is the score signal (matrix completion coincides with it there); we shrink it toward the PKPS estimate by a precision weight $\sigma_q^2/(\sigma_q^2 + e_{PKPS}^2)$—the sample's noise variance over its total disagreement with PKPS—so the own sample dominates as queries grow and the embedding dominates as noise grows. Both quantities are read off the observed queries—the sample noise variance estimated as $\hat{p}(1-\hat{p})/m$ and the prior error from its depth-weighted disagreement with the sample—so the weight never touches the held-out full scores and the score baselines see exactly the same information.

3.3 Query efficiency

PKPS is designed to reach a target prediction accuracy from fewer evaluated queries than paired DKPS: by bridging responses to similar queries it uses evidence that DKPS, limited to shared queries, discards. We state the expected gain as a conjecture and leave a formal analysis—in the style of the DKPS query-efficiency bound [Helm et al., 2026]—to future work; Section 4.2 is its empirical counterpart.

Conjecture 1 (Query efficiency of PKPS)**.** *Fix a benchmark score function that is Lipschitz on the perspective space and a target error ε. For a query design $\mathbf{Q} = (Q_1, \dots, Q_n)$ with cross-model overlap ρ (the fraction of query-kernel mass shared across models), the number of queries per model that PKPS needs to reach error ε is at most that of paired DKPS, and the gap is increasing as $\rho \to 0$; the two are closest in the fully paired limit $\rho = 1$.*

4 Experiments

A real evaluation matrix is governed by four observation levers, which structure our experiments. The first is the one DKPS cannot handle: (i) *pairing*—the fraction ρ of each cell's queries that are shared across models, from fully paired (ρ=1) to fully unpaired (ρ=0). The other three set how sparse and noisy the observations are: (ii) *task coverage*—the fraction of (model, task) cells run at all; (iii) *queries per cell*—how many queries a run uses, which sets the per-cell measurement *noise*; and (iv) *cohort size*—the number of models. The all-tasks-covered, few-query limit is efficient benchmarking (denoising) and the zero-query limit is benchmark completion—two applications of the same embedding.

4.1 Synthetic data

We generate n models with latent ability vectors v_i and T tasks with latent vectors t_k, scores $v_i \cdot t_k + \varepsilon$, and queries drawn near each task center. Each model answers M_{ij} queries per task, of which $m_{ii'}$ are

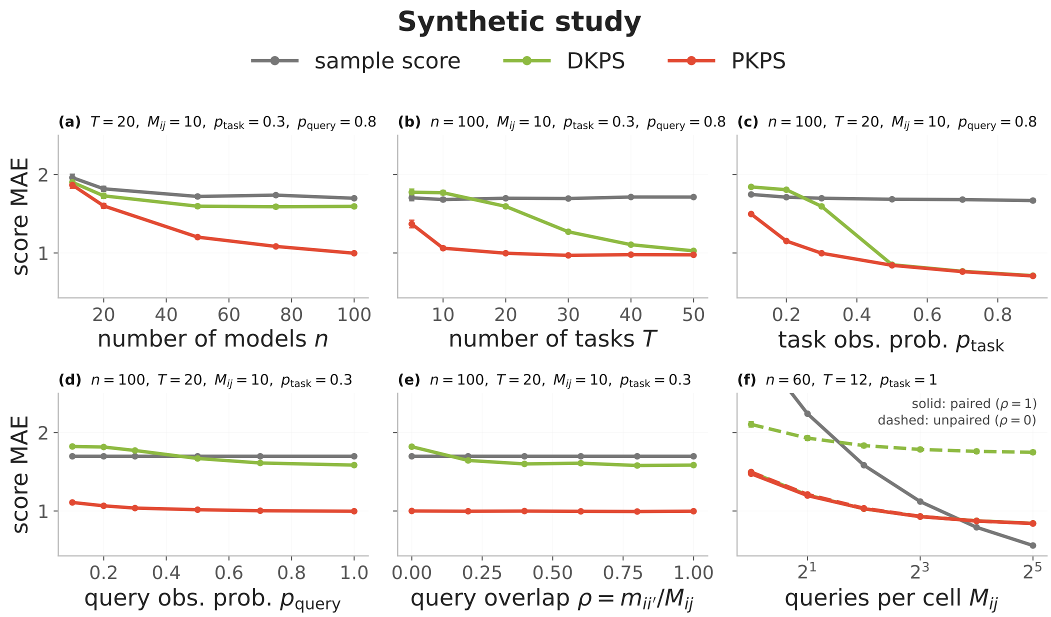

Figure 2: Synthetic study (score MAE, ± 1 SEM over 50 seeds). DKPS (identity query kernel, green) vs. PKPS (RBF, red), with the score-only sample baseline (gray); each panel sweeps one parameter with the rest fixed (shown in its title). **(a)–(e) complete held-out cells** from observed responses—cohort n, tasks T, task coverage p_{task}, query coverage p_{query}, and cross-model overlap $\rho = m_{ii'}/M_{ij}$. PKPS (CV-selected bandwidth) beats DKPS throughout—by a wide margin even when mostly paired (a–c), since it bridges across tasks as well as within—and the gap is largest when unpaired or sparse (d, e), where in (e) DKPS collapses toward the floor as $\rho \to 0$ while PKPS stays flat. **(f) denoises observed cells**: each cell is measured with M_{ij} noisy queries and we predict its true score, at paired (ρ=1, solid) and unpaired (ρ=0, dashed). The raw sample falls $\propto 1/\sqrt{M_{ij}}$; smoothing over the perspective denoises well below it at small budgets—PKPS paired or unpaired, DKPS only paired—until the sample becomes precise at the full budget.

shared with another model (overlap $\rho = m_{ii'}/M_{ij}$); a task is observed with probability p_{task} and a query with probability p_{query}. We predict held-out (model, task) scores and compare DKPS (identity query kernel) against PKPS (RBF), with the score-only sample baseline (predict a held-out cell by its task's observed mean) as a floor (Figure 2).

PKPS—selecting its query bandwidth by the same leak-free cross-validation we use on real data—beats DKPS across *every* lever, and both beat the sample floor: bridging responses to similar queries (across tasks as well as within) supplies model-similarity signal that exact-match DKPS cannot reach, so the margin is wide even where queries are mostly paired (panels a–c). The gap is largest where pairing is scarce: as query coverage p_{query} drops (d) DKPS degrades while PKPS stays flat, and sweeping the overlap ρ directly (e), DKPS collapses toward the sample floor as $\rho \to 0$ while PKPS is essentially flat—it does not rely on pairing.

Panel (f) is the *denoising* form of the same problem: each cell is now *observed* with M_{ij} noisy queries (sample noise $\propto 1/\sqrt{M_{ij}}$) and we predict its true score. The raw sample mean (gray) falls like $1/\sqrt{M_{ij}}$; smoothing those noisy scores over the perspective denoises well below it while measurement noise dominates—PKPS reaches at a single query the accuracy the raw sample needs roughly five queries to match, and does so whether queries are paired or not, while DKPS denoises only when paired (unpaired DKPS, dashed, barely improves). Only once the budget is large is the raw sample precise enough to overtake the embedding's residual bias. This is the empirical counterpart of Conjecture 1.

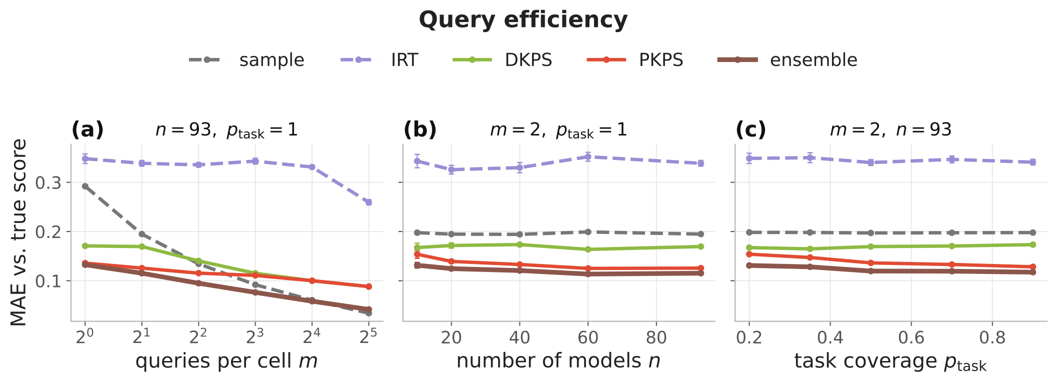

Figure 3: **Query efficiency.** Each model answers m unpaired queries per *observed* cell; predict its true full-eval score (MAE, ± 1 SEM). PKPS (red) and the ensemble (brown) denoise far below the raw sample (grey) at small budgets and across cohort size (b) and coverage (c); DKPS (green) trails because the queries are unpaired and IRT (purple) cannot fit at all.

4.2 Real data

We evaluate on one realistic, heterogeneous benchmark, studying the two applications the unpaired embedding enables: making evaluation more *query-efficient* (denoising cheap, few-query runs) and completing *missing* cells. Both are *unpaired* regimes—each model answers its own queries, so DKPS's identity matching and IRT's shared-anchor fit degrade while PKPS bridges similar queries. All real-data numbers are means over 16 seeds, and PKPS selects its query bandwidth by cross-validation per run. The suite is 18 tasks across four datasets with very different response types—MATH (7 math subjects), WMT-14 (5 translation directions), MedQA (medical multiple-choice), and LegalBench (5 legal classification subsets)—on the 93 models evaluated on all four. MATH and WMT have rich free-text responses; MedQA and LegalBench responses are single tokens for which text embeddings are degenerate, so we encode each dataset's responses natively (Gemini embeddings vs. one-hot answers) in disjoint blocks. With a linear k_R this makes cross-dataset response similarity exactly zero, and a single shared query embedding with a within-domain bandwidth keeps k_Q near zero across domains: the product kernel factorizes by domain, never comparing a proof to a legal label.

Application 1: query efficiency. Because the embedding captures response *style*, which is stable even when the per-cell sample mean is noisy, it denoises cheap evaluations. Each model answers m queries per cell (sampled independently—so this is itself an unpaired regime, and IRT cannot fit); we predict the true full-eval score (Figure 3). At a small budget the embedding is far more accurate than the raw mean: at $m=1$, PKPS scores 0.136 and the ensemble 0.133 versus the sample's 0.292; PKPS at one query roughly matches the sample at four. As the budget grows the raw mean catches up and eventually overtakes the embedding's residual bias (0.034 vs. 0.088 at $m=32$); the leak-free ensemble tracks the better channel to within a few thousandths. The advantage is robust across cohort size and task coverage (panels b, c). Per cell, the denoising win is broad but tail-driven: the ensemble beats the raw sample on most cells, with the largest gains at the smallest budgets and on the rich-response datasets (Figure 4).

Application 2: matrix completion. The same embedding also predicts cells that were *never run*. We observe a fraction p_{task} of cells (each at depth p_{query}) and predict the missing ones, against a low-rank matrix-completion baseline (the sample does not apply—a missing cell has no sample). The contrast is the cohort: low-rank completion needs many rows, so when models are scarce it fails for lack of them while the embedding, which needs none, remains accurate—at $n=10$, completion

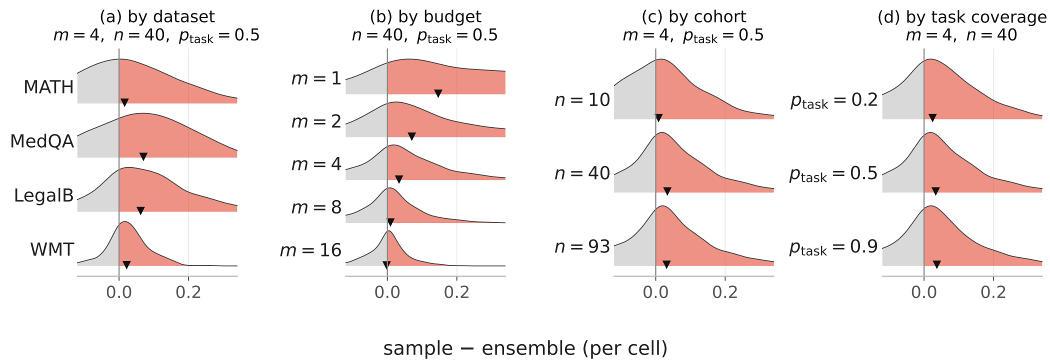

Figure 4: **Conditional breakdown of the query-efficiency win.** Per-cell improvement $\Delta =$ sample $-$ ensemble MAE ($\Delta > 0$, red, the ensemble wins) stacked by dataset, by per-cell budget m, by cohort n, and by task coverage p_{task}, holding the other levers at their sweep median (m=4, n=40, p_{task}=0.5). The win is broad but tail-driven—largest at the smallest budgets and on the rich-response datasets—with the per-row mean (▼) positive throughout.

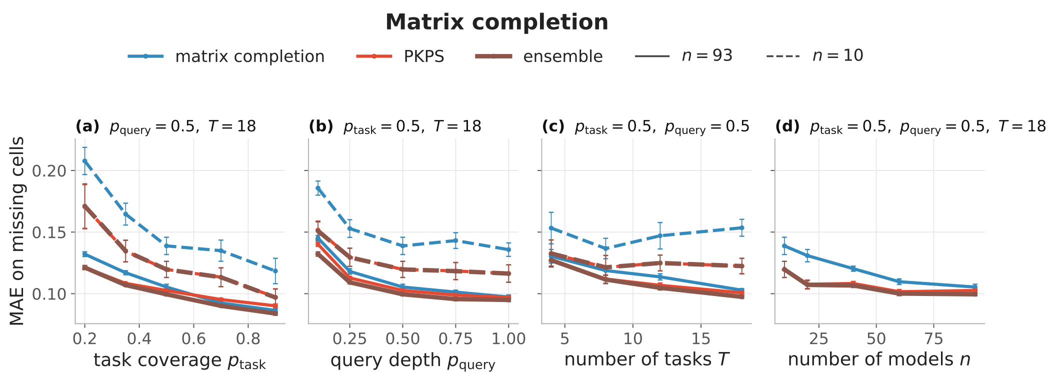

Figure 5: **Matrix completion.** Observe a fraction of (model, task) cells, predict the *missing* ones (MAE on missing cells, ± 1 SEM). Colour is method (low-rank completion blue, PKPS red, ensemble brown); line style is cohort (solid n=93, dashed n=10). The embedding rescues the small-cohort regime where low-rank completion collapses for lack of rows, and ties it at full cohort.

is 0.153 versus PKPS's 0.127 (Figure 5, dashed). At full cohort (n=93, solid) PKPS slightly edges completion (0.104 vs. 0.107) and the ensemble is best (0.100). Even in the regime most favorable to it—a dense, low-rank matrix—completion is matched or slightly beaten; the embedding's clear advantage is the sparse-cohort regime completion cannot reach. The per-cell win over completion is modest and tail-driven—completion is strong on this low-rank suite—but stays positive in the mean across datasets, cohorts, coverage, and query depth (Figure 6).

Summary. Table 1 collects the two applications at a hard operating point each. Both are unpaired regimes, so the same product-kernel embedding that DKPS and IRT cannot match denoises cheap evaluations and completes missing cells—PKPS is present and competitive in both, and the ensemble, which folds in a cell's own sample, is best wherever that sample exists.

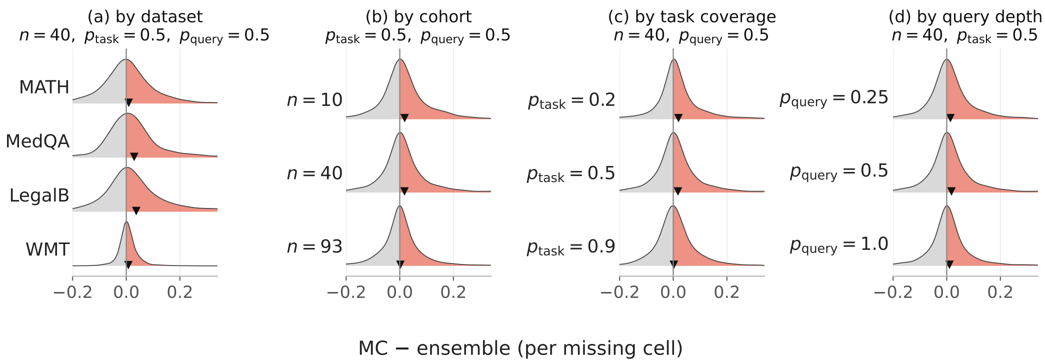

Figure 6: **Conditional breakdown of the completion win.** Per-missing-cell improvement $\Delta =$ matrix-completion $-$ ensemble MAE ($\Delta > 0$, red, the ensemble wins) stacked by dataset, by cohort n, by task coverage p_{task}, and by query depth p_{query}, holding the other levers at their sweep median (n=40, p_{task}=0.5, p_{query}=0.5). The win is modest and tail-driven on this low-rank suite—the regime most favorable to completion—but the per-row mean (▼) stays positive across conditions.

Table 1: Suite MAE vs. the full-eval score at one hard operating point per regime (lower is better; best in bold). "—" marks a baseline that does not apply to that regime: the sample mean and IRT need a query a cell never ran (completion), and matrix completion needs a cell that was never run (the observed-cell regimes). PKPS applies in every regime.

Method	**Query-efficient** (m=1/cell)	**Completion** (missing, n=10)
sample mean	0.292	—
IRT	0.348	—
matrix completion	—	0.153
DKPS	0.171	—
PKPS	0.136	**0.127**
ensemble	**0.133**	**0.127**

5 Discussion

Real leaderboards are assembled organically: models are run on whatever query subsets their authors chose, so the responses that benchmark estimation must compare are *unpaired*. DKPS and item-response models need a shared block of identical queries and collapse without it. PKPS removes that requirement with a product kernel $k_Q \cdot k_R$ that compares responses to *similar* queries, recovering paired DKPS exactly in its $\sigma \to 0$ limit and scaling to a heterogeneous 18-task suite through a block-diagonal kernel. Our central finding is that benchmark estimation remains accurate under unpaired evaluation: as the shared fraction $\rho \to 0$, DKPS and IRT degrade sharply while PKPS holds. That capability yields two practical applications from the same embedding—it denoises cheap, few-query evaluations (query efficiency) and predicts cells that were never run (matrix completion), and ensembled with a cell's own sample where one exists it is never the worse choice. None of this assumes pairing or exhaustive per-cell evaluation, which is exactly what organically-assembled leaderboards cannot provide.

References

Aranyak Acharyya, Michael W. Trosset, Carey E. Priebe, and Hayden S. Helm. Consistent estimation of generative model representations in the data kernel perspective space. *arXiv preprint* *arXiv:2409.17308*, 2024.

Aranyak Acharyya, Joshua Agterberg, Youngser Park, and Carey E. Priebe. Concentration bounds on response-based vector embeddings of black-box generative models. *arXiv preprint arXiv:2511.08307*, 2025.

Andrew M. Bean et al. Scales++: Compute efficient evaluation subset selection with cognitive scales embeddings. *arXiv preprint arXiv:2510.26384*, 2025.

Emmanuel J. Candès and Benjamin Recht. Exact matrix completion via convex optimization. *Foundations of Computational Mathematics*, 9(6):717–772, 2009.

Nan Chen, Hayden Helm, Youngser Park, Carey Priebe, and Soledad Villar. Extracting information from fine-tuned weights. In *Non-Euclidean Foundation Models: Advancing AI Beyond Euclidean Frameworks*, 2025.

Brandon Duderstadt, Hayden S. Helm, and Carey E. Priebe. Comparing foundation models using data kernels. *arXiv preprint arXiv:2305.05126*, 2023.

Hayden Helm, Brandon Duderstadt, Youngser Park, and Carey E. Priebe. Tracking the perspectives of interacting language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

Hayden Helm, Aranyak Acharyya, Youngser Park, Brandon Duderstadt, and Carey Priebe. Statistical inference on black-box generative models in the data kernel perspective space. In *Findings of the Association for Computational Linguistics (ACL)*, 2025.

Hayden Helm, Ben Johnson, and Carey E. Priebe. Query-efficient model evaluation using cached responses. *arXiv preprint arXiv:2605.07096*, 2026.

Eliahu Horwitz, Nitzan Kurer, Jonathan Kahana, Liel Amar, and Yedid Hoshen. We should chart an atlas of all the world's models. *arXiv preprint arXiv:2503.10633*, 2025.

Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The platonic representation hypothesis. *arXiv preprint arXiv:2405.07987*, 2024.

Andrew Ilyas, Sung Min Park, Logan Engstrom, Guillaume Leclerc, and Aleksander Madry. Datamodels: Predicting predictions from training data. *arXiv preprint arXiv:2202.00622*, 2022.

Alex Kipnis, Konstantinos Voudouris, Luca M. Schulze Buschoff, and Eric Schulz. metabench: A sparse benchmark of reasoning and knowledge in large language models. *arXiv preprint arXiv:2407.12844*, 2024.

Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.

Yang Li, Jie Ma, Miguel Ballesteros, Yassine Benajiba, and Graham Horwood. Active evaluation acquisition for efficient llm benchmarking. *arXiv preprint arXiv:2410.05952*, 2024.

Rahul Mazumder, Trevor Hastie, and Robert Tibshirani. Spectral regularization algorithms for learning large incomplete matrices. *Journal of Machine Learning Research*, 2010.

Yotam Perlitz, Elron Bandel, Ariel Gera, Ofir Arviv, Liat Ein-Dor, Eyal Shnarch, Noam Slonim, Michal Shmueli-Scheuer, and Leshem Choshen. Efficient benchmarking (of language models). In *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinybenchmarks: Evaluating llms with fewer examples. In *International Conference on Machine Learning (ICML)*, 2024.

Ameya Prabhu, Vishaal Udandarao, Philip Torr, Matthias Bethge, Adel Bibi, and Samuel Albanie. Efficient lifelong model evaluation in an era of rapid progress. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Georg Rasch. *Probabilistic Models for Some Intelligence and Attainment Tests*. Danish Institute for Educational Research, 1960.

Warren S Torgerson. Multidimensional scaling: I. theory and method. In *Psychometrika*, 1952.

Rajan Vivek, Kawin Ethayarajh, Diyi Yang, and Douwe Kiela. Anchor points: Benchmarking models with much fewer examples. In *EACL*, 2024.

Yuchen Zeng and Dimitris Papailiopoulos. You don't need to run every eval. *arXiv preprint arXiv:2606.24020*, 2026.